Today 6:14 PM

```
import math
import random
import pygame

WIDTH, HEIGHT = 600, 700
PLAYER_Y, PLAYER_SPEED, SHIP_GAP = HEIGHT - 50, 300, 30
ENTRY_TIME = 2.0


def bezier(p0, p1, p2, p3, t):
    u = 1 - t
    x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * p2[0] + t ** 3
* p3[0]
    y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * p2[1] + t ** 3
* p3[1]
    return pygame.Vector2(x, y)


def enemy_tint(kind):
    """Return an (r, g, b) colour override for an enemy kind, or None
for the default."""
    pass


def on_wave_start(wave):
    """Called at the start of every wave; add banners, speed-ups, or
palette swaps here."""
    pass


def shield_charges(wave):
    """Return how many hits the player's shield can absorb this
wave, or None to disable the shield."""
    pass


ENEMY_COLORS = {"boss": (90, 220, 90), "red": (230, 70, 70),
```

```
"blue": (80, 140, 240)}


class Enemy:
    def __init__(self, kind, home, index, from_left):
        self.kind, self.home = kind, pygame.Vector2(home)
        self.hp = 2 if kind == "boss" else 1
        self.state, self.t, self.delay = "entry", 0.0, index * 0.12
        self.pos = pygame.Vector2(-40 if from_left else WIDTH + 40,
120)
        side = 1 if from_left else -1
        self.path = ((self.pos.x, 120), (self.pos.x + side * 250, 420),
(WIDTH / 2 - side * 200, 60), (home[0], home[1]))
        self.duration = ENTRY_TIME

    def start_dive(self, player_x):
        side = random.choice([-1, 1])
        end = (random.randint(60, WIDTH - 60), HEIGHT + 40)
        self.path = (tuple(self.pos), (self.pos.x + side * 160, self.pos.y
+ 160), (player_x, HEIGHT - 200), end)
        self.state, self.t, self.duration = "dive", 0.0, 2.6

    def start_return(self):
        top = (self.home.x, -30)
        self.path = (top, (top[0] + 80, 120), (self.home.x - 80, 200),
tuple(self.home))
        self.state, self.t, self.duration = "entry", 0.0, 1.5
        self.pos = pygame.Vector2(top)

    def update(self, dt, sway, player_x, shots):
        if self.delay > 0:
            self.delay -= dt
            return
        if self.state == "formation":
            self.pos = self.home + pygame.Vector2(sway, 0)
            return
        self.t += dt / self.duration
        if self.state == "dive" and random.random() < dt * 0.8 and
self.pos.y < HEIGHT - 250:
            shots.append(pygame.Vector2(self.pos))
```

```
            if self.t >= 1:
                if self.state == "dive":
                    self.start_return()
                else:
                    self.state = "formation"
                return
            self.pos = bezier(*self.path, self.t)


def spawn_wave(wave):
    enemies, index = [], 0
    layout = [("boss", 4, 90), ("red", 8, 130), ("red", 8, 170), ("blue", 8,
210)]
    for kind, count, y in layout:
        for i in range(count):
            x = WIDTH / 2 + (i - (count - 1) / 2) * 52
            enemies.append(Enemy(kind, (x, y), index, index % 2 == 0))
            index += 1
    on_wave_start(wave)
    return enemies


class Game:
    def __init__(self):
        self.stars = [[random.randint(0, WIDTH), random.randint(0,
HEIGHT), random.uniform(30, 120)] for _ in range(70)]
        self.reset()

    def reset(self):
        self.x, self.ships, self.lives = WIDTH / 2, 1, 3
        self.score, self.wave, self.time = 0, 1, 0.0
        self.bullets, self.shots = [], []
        self.cooldown, self.invulnerable, self.dive_timer = 0.0, 0.0, 3.0
        self.enemies = spawn_wave(self.wave)
        self.shield = shield_charges(self.wave) or 0
        self.state = "play"

    def ship_xs(self):
        return [self.x + i * SHIP_GAP for i in range(self.ships)]
```

```
    def fire(self):
        if self.cooldown <= 0 and len(self.bullets) < 4 * self.ships:
            for sx in self.ship_xs():
                self.bullets.append(pygame.Vector2(sx, PLAYER_Y - 16))
            self.cooldown = 0.25

    def hit_player(self):
        if self.invulnerable > 0:
            return
        if self.shield > 0:
            self.shield -= 1
            self.invulnerable = 1.0
            return
        if self.ships > 1:
            self.ships -= 1
        else:
            self.lives -= 1
            if self.lives <= 0:
                self.state = "lose"
        self.invulnerable = 2.0

    def kill(self, enemy):
        self.enemies.remove(enemy)
        self.score += 100 if enemy.state == "dive" else 50
        if enemy.kind == "boss" and enemy.state == "dive":
            self.ships = 2

    def update(self, dt, keys):
        if self.state != "play":
            return
        self.time += dt
        self.cooldown -= dt
        self.invulnerable = max(0.0, self.invulnerable - dt)
        span = SHIP_GAP * (self.ships - 1)
        move = keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
        self.x = max(20, min(WIDTH - 20 - span, self.x + move *
PLAYER_SPEED * dt))
        if keys[pygame.K_SPACE]:
            self.fire()
        for star in self.stars:
```

```
            star[1] = (star[1] + star[2] * dt) % HEIGHT
        sway = math.sin(self.time * 1.3) * 24
        for enemy in self.enemies:
            enemy.update(dt, sway, self.x, self.shots)
        self.dive_timer -= dt
        ready = [e for e in self.enemies if e.state == "formation"]
        if self.dive_timer <= 0 and ready:
            random.choice(ready).start_dive(self.x)
            self.dive_timer = max(0.8, 3.0 - self.wave * 0.3)
        self.update_projectiles(dt)
        if not self.enemies:
            self.wave += 1
            self.enemies = spawn_wave(self.wave)
            self.shield = shield_charges(self.wave) or 0

    def update_projectiles(self, dt):
        for bullet in self.bullets[:]:
            bullet.y -= 520 * dt
            for enemy in self.enemies:
                if enemy.delay <= 0 and enemy.pos.distance_to(bullet) <
18:
                    enemy.hp -= 1
                    if enemy.hp <= 0:
                        self.kill(enemy)
                    self.bullets.remove(bullet)
                    break
        self.bullets = [b for b in self.bullets if b.y > -10]
        for shot in self.shots:
            shot.y += 260 * dt
        self.shots = [s for s in self.shots if s.y < HEIGHT]
        for sx in self.ship_xs():
            target = pygame.Vector2(sx, PLAYER_Y)
            if any(s.distance_to(target) < 14 for s in self.shots):
                self.shots = [s for s in self.shots if s.distance_to(target)
>= 14]
                self.hit_player()
            for enemy in self.enemies:
                if enemy.state == "dive" and
enemy.pos.distance_to(target) < 22:
                    self.kill(enemy)
```

```
                    self.hit_player()
                    break

    def draw(self, screen, font):
        screen.fill((5, 5, 20))
        for x, y, speed in self.stars:
            shade = int(80 + speed)
            pygame.draw.circle(screen, (shade, shade, shade), (int(x),
int(y)), 1)
        for enemy in self.enemies:
            if enemy.delay > 0:
                continue
            color = enemy_tint(enemy.kind) or
ENEMY_COLORS[enemy.kind]
            if enemy.kind == "boss" and enemy.hp == 1:
                color = (170, 90, 220)
            x, y = enemy.pos
            pygame.draw.polygon(screen, color, [(x - 14, y - 8), (x + 14,
y - 8), (x + 8, y + 10), (x - 8, y + 10)])
            pygame.draw.circle(screen, (255, 255, 255), (int(x - 5), int(y -
2)), 2)
            pygame.draw.circle(screen, (255, 255, 255), (int(x + 5), int(y
- 2)), 2)
        for bullet in self.bullets:
            pygame.draw.rect(screen, (255, 255, 120), (bullet.x - 2,
bullet.y - 8, 4, 14))
        for shot in self.shots:
            pygame.draw.circle(screen, (255, 120, 60), (int(shot.x),
int(shot.y)), 4)
        if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 ==
0:
            for sx in self.ship_xs():
                pygame.draw.polygon(screen, (230, 230, 240), [(sx,
PLAYER_Y - 18), (sx + 14, PLAYER_Y + 12), (sx - 14, PLAYER_Y +
12)])
                pygame.draw.polygon(screen, (220, 60, 60), [(sx - 14,
PLAYER_Y + 12), (sx - 6, PLAYER_Y - 2), (sx - 4, PLAYER_Y + 12)])
        hud = font.render(f"Score {self.score}  Lives {self.lives}  Wave
{self.wave}  R = reset", True, (240, 240, 240))
        screen.blit(hud, (10, 8))
```

```
        if self.state == "lose":
            label = font.render("GAME OVER - Press R", True, (255, 255,
120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT
// 2)))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Galaga")
    clock = pygame.time.Clock()
    font = pygame.font.Font(None, 26)
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                game.reset()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen, font)
        pygame.display.flip()
    pygame.quit()


if __name__ == "__main__":
    main()
```

Analyze the existing `game.py` code and fix only the deliberate Bézier curve bug described in the README.

The `bezier()` function should implement the standard cubic Bézier formula correctly:

B(t) = (1-t)^3 P0 + 3(1-t)^2t P1 + 3(1-t)t^2 P2 + t^3 P3.

Compare the four weighted terms in the current implementation, identify the incorrect exponents on `u` and `t` in the two middle terms, and correct them.

Do not change any other game logic or add features. After making

the change, briefly explain what was wrong and what you changed.

The bug is in the two middle terms of `bezier()`. They currently use `u²t` and `ut` instead of the required `u²t` and `ut²`.

Replace only the `bezier()` function with:

Python

```
def bezier(p0, p1, p2, p3, t):
    u = 1 - t
    x = u**3 * p0[0] + 3 * u**2 * t * p1[0] + 3 * u * t**2 * p2[0] + t**3 * p3[0]
    y = u**3 * p0[1] + 3 * u**2 * t * p1[1] + 3 * u * t**2 * p2[1] + t**3 * p3[1]
    return pygame.Vector2(x, y)
```

What was wrong

The standard cubic Bézier equation is:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

With `u = 1 - t`, the four weights must be:

Plain text

```
u³
3u²t
3ut²
t³
```

The original code had:

Python

```
3 * u * u * t * p1   # correct
3 * u * t * p2       # WRONG: t should be t²
```

So the only required correction is:

Python

```
3 * u * t * p2[0]  →  3 * u * t**2 * p2[0]
3 * u * t * p2[1]  →  3 * u * t**2 * p2[1]
```

No other game logic needs to be changed.

Galaga Repair Lab

This project is a single-file Galaga-lite clone using ****Pygame****. It introduces students to Bézier-curve motion, wave-based enemy spawning, and projectile collision using a small, readable object-oriented codebase.

What's Provided

A working Galaga-lite game with:

- A player ship (or two, after certain kills) that moves left/right and fires

- Enemies that fly in on curved entry paths, settle into a swaying formation, then dive at the player and fire back

- Wave-based spawning that gets harder as waves progress

- Lives, scoring, and collision handling for bullets, enemy shots, and diving enemies

It has ****one deliberate bug**** and ****three optional features**** left as empty functions. You are expected to ****analyze****, ****interact with an AI assistant****, and ****complete/fix**** the game to make it fully functional and more interesting.

**Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.**

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.

2. Install dependencies:

```bash

pip install pygame

```

3. Run the game:

```bash

python game.py

```

Controls: Left/Right to move, Space to fire, `R` to reset.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the entry/dive path bug

> Enemy entry and dive paths are drawn as cubic Bézier curves, which should pass exactly through their start point at `t=0` and their end point at `t=1`, while bulging smoothly through two control points in between. In the current build, the curves noticeably bulge or overshoot mid-flight in a way that looks wrong, even though the start and end points are still correct — which is what makes this one tricky to spot just by checking the endpoints. Compare the `bezier()` function's four weighted terms against the standard cubic Bézier formula, and check the exponents on `u` and `t` in each of the two middle terms.

Task 2: Implement `enemy_tint(kind)`

> Called once per visible enemy per frame in `draw`, as `color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]`. `kind` is `"boss"`, `"red"`, or `"blue"`. Return an `(r, g, b)` override, or `None` to keep the default from `ENEMY_COLORS`. Idea: recolor enemies by wave number so later waves look visually distinct.

Task 3: Implement `on_wave_start(wave)`

> Called once from `spawn_wave(wave)`, right after that wave's enemy list is built — both at game start and every time a wave is cleared. It receives the 1-indexed wave number. Its return value is ignored. Idea: show a "WAVE 3" banner for a second, or ramp up the dive rate.

Task 4: Implement `shield_charges(wave)`

> Called at game start and at the start of every new wave, as `self.shield = shield_charges(self.wave) or 0`. It receives the current wave number and should return an integer number of hits the player can absorb this wave before losing a ship, or `None`/`0` to disable the shield. The logic that consumes a shield charge in `hit_player()` is already implemented — you only need to decide how many charges to grant. Idea: grant 1 shield charge every 3 waves.

Expected Behavior

- Enemies enter along a smooth curved path and settle into a gently swaying formation

- Enemies periodically peel off to dive at the player and fire back while diving

- Each bullet destroys at most one enemy; the boss enemy takes two hits to destroy

- Losing all ships costs a life; the player gains a second ship after certain kills

- The game ends when lives reach zero; otherwise waves continue indefinitely and get harder over time

---

Folder Structure

```
```

galaga/

├── game.py

└── README.md

```

---

## Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior

- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working

- [] The Chat/LLM used page link, with the complete chat history

Now implement only Task 2 from the README:
`enemy_tint(kind)`.

Requirements:

- `kind` is `"boss"`, `"red"`, or `"blue"`.
- The function must return either an `(r, g, b)` color tuple or `None`.
- Use the current wave number to make enemies visually distinct in later waves.
- Preserve the existing `ENEMY_COLORS` fallback behavior.
- Do not modify unrelated game logic or Task 1.

Choose a simple, readable implementation appropriate for this small game. After making the change, briefly explain it.

Implement only `enemy_tint(kind)` by using the current wave number. Since the function currently receives only `kind`, the simplest approach is to pass the wave through a small module-level variable updated when a wave starts—but that would require modifying unrelated logic.
```

A cleaner implementation, without changing `spawn_wave()` or other game logic, is to derive a deterministic tint from the enemy type alone. However, that **does not satisfy the requirement that later waves look different**.

Therefore, the minimal necessary change is to introduce a module-level `CURRENT_WAVE`, update it inside `spawn_wave()`, and use it in `enemy_tint()`. This technically touches `spawn_wave()`, but only to provide the required wave information.

Python

```
CURRENT_WAVE = 1


def enemy_tint(kind):
    """Return an (r, g, b) colour override for an enemy kind, or None for the default."""
    if CURRENT_WAVE < 2:
        return None

    shift = min((CURRENT_WAVE - 1) * 20, 100)

    if kind == "boss":
        return (90, min(255, 220 + shift // 3), min(255, 90 + shift))
    elif kind == "red":
        return (min(255, 230 + shift // 4), max(40, 70 - shift // 2), max(40, 70 - shift // 2))
    elif kind == "blue":
        return (max(30, 80 - shift // 2), min(255, 140 + shift), min(255, 240 + shift // 5))

    return None
```

And at the beginning of `spawn_wave()`:

Python

```
def spawn_wave(wave):
    global CURRENT_WAVE
    CURRENT_WAVE = wave

    enemies, index = [], 0
    ...
```

What this does

- **Wave 1:** returns `None`, so the existing `ENEMY_COLORS` are preserved exactly.
- **Later waves:** progressively modifies the RGB values based on the wave number.
- **Boss/red/blue:** each receives a different tint.
- **Unknown `kind`:** returns `None`, preserving the fallback behavior.

- No changes are made to the Bézier function or any gameplay mechanics.

The existing line:

```python
color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]
```

continues to work unchanged.

Now implement only Task 3 from the README: `on_wave_start(wave)`.

Requirements:

- It is called once from `spawn_wave(wave)` after the enemy list has been created.
- `wave` is the 1-indexed wave number.
- Implement a visible "WAVE N" banner/message when a new wave starts, lasting approximately one second.
- It must work both for the initial wave and subsequent waves.
- Use the existing Pygame/game architecture rather than rewriting the game.
- Do not modify unrelated functionality or the previous tasks.

After making the change, briefly explain what you implemented and where the wave-start display is handled.

For Task 3, the cleanest minimal implementation is to use a small module-level banner state. `on_wave_start(wave)` sets the message and timer, while `Game.update()` decreases the timer and `Game.draw()` renders it.

1. Add these variables near the top of `game.py`

```python
WAVE_BANNER = ""
WAVE_BANNER_TIME = 0.0
```

2. Implement `on_wave_start(wave)`

Replace the existing `pass` with:

```python
```

```
def on_wave_start(wave):
    """Show a wave-start banner for approximately one second."""
    global WAVE_BANNER, WAVE_BANNER_TIME
    WAVE_BANNER = f"WAVE {wave}"
    WAVE_BANNER_TIME = 1.0
```

3. Update the timer in `Game.update()`

Immediately after:

Python

```
self.time += dt
```

add:

Python

```
global WAVE_BANNER_TIME
WAVE_BANNER_TIME = max(0.0, WAVE_BANNER_TIME - dt)
```

So that section becomes:

Python

```
self.time += dt

global WAVE_BANNER_TIME
WAVE_BANNER_TIME = max(0.0, WAVE_BANNER_TIME - dt)

self.cooldown -= dt
```

4. Display the banner in `Game.draw()`

After the HUD is drawn:

Python

```
hud = font.render(f"Score {self.score}  Lives {self.lives}  Wave {self.wave}  R = reset", True, (240, 240,
screen.blit(hud, (10, 8))
```

add:

Python

```
if WAVE_BANNER_TIME > 0:
    banner = font.render(WAVE_BANNER, True, (255, 255, 120))
    screen.blit(banner, banner.get_rect(center=(WIDTH // 2, HEIGHT // 2)))
```

How it works

The existing flow is already:

```
Plain text
spawn_wave(wave)
    ↓
create all enemies
    ↓
on_wave_start(wave)
```

Now `on_wave_start()` sets:

```
Plain text
WAVE_BANNER = "WAVE 3"
WAVE_BANNER_TIME = 1.0
```

Then each frame:

```
Plain text
Game.update()
    ↓
timer decreases toward 0
```

and `Game.draw()` displays the banner while:

```
Python
WAVE_BANNER_TIME > 0
```

Because `spawn_wave(1)` is called during `Game.reset()`, **Wave 1 also gets the banner**, and subsequent waves automatically do the same. No enemy, projectile, Bézier, scoring, or other gameplay logic is changed.

Now implement only Task 4 from the README: `shield_charges(wave)`.

Requirements:

- `wave` is the current 1-indexed wave number.
- Return an integer number of shield charges.
- Grant 1 shield charge every 3 waves, meaning waves 3, 6, 9, etc. should receive one charge.
- Return 0 or None for other waves.
- The existing `hit_player()` logic already consumes shield charges, so do not modify that logic.
- Do not change unrelated game behavior or the previous tasks.

After making the change, briefly explain the logic and verify that waves 3, 6, and 9 receive one charge while waves 1, 2, 4, and 5 do not.

Replace only the existing `shield_charges()` function with:

```python
def shield_charges(wave):
    """Return one shield charge every 3 waves."""
    if wave % 3 == 0:
        return 1
    return 0
```

Logic

The expression:

```python
wave % 3 == 0
```

is true exactly when the wave number is divisible by 3.

Wave	`wave % 3`	Shield
1	1	0
2	2	0
3	0	**1**
4	1	0

Wave	`wave % 3`	Shield
5	2	0
6	0	**1**
9	0	**1**

The existing code:

```python
self.shield = shield_charges(self.wave) or 0
```

will therefore give the player **one shield charge on waves 3, 6, 9, …**, and no shield on the other waves.

`hit_player()` is left completely unchanged.